

Real-time Vision-Based Virtual Piano

Yucheng Su
2024011306

Xiaowen Lou
2024011321

Abstract

We present a real-time virtual piano system that enables users to play music on a flat surface using only hand gestures captured by a consumer-grade RGB-D camera. The system employs MediaPipe for 21-point hand landmark detection, followed by a multi-stage fingertip stabilization pipeline combining exponential moving average (EMA) smoothing with Lucas-Kanade optical flow anchoring. A velocity-sensitive six-state finite state machine detects keystroke events by analyzing relative finger motion trajectories, distinguishing intentional strikes from passive hand movements. We further investigate the feasibility of iPhone LiDAR depth sensing for contact detection and demonstrate that consumer-grade Time-of-Flight sensors provide insufficient depth coverage on downward-pointing fingertips, motivating a depth-assist architecture where depth information serves as a negative gate rather than a primary trigger. The system supports extended features including velocity dynamics, scale/mode switching, chord mode, sustain pedal simulation, metronome, and MIDI export. We demonstrate the system’s reliability through a complete performance of *Twinkle Twinkle Little Star* with almost no missed or incorrect notes.

1 Introduction

Vision-based musical instrument interaction offers an accessible alternative to physical keyboards, enabling music creation without specialized hardware. However, building a playable virtual piano from camera input alone presents several fundamental challenges:

- **Tracking jitter:** Hand landmark detectors such as MediaPipe exhibit frame-to-frame positional noise on the order of 10–20 pixels at the 95th percentile, which is comparable in magnitude to intentional keystroke movements.
- **Strike disambiguation:** Distinguishing a deliberate downward tap from passive finger drift, hand translation, or detection noise requires robust motion analysis rather than simple threshold crossing.
- **Real-time constraints:** At approximately 30 frames per second, the system has only ~ 33 ms between frames to detect, stabilize, and classify finger motion—fast sequential keystrokes may span only 2–3 frames.
- **Depth sensor limitations:** Consumer-grade LiDAR sensors (e.g., iPhone TrueDepth) exhibit severe coverage gaps on small, angled targets such as downward-pointing fingertips, precluding depth-only contact detection.

We address these challenges with the following contributions:

1. A **multi-stage fingertip stabilization pipeline** combining EMA temporal smoothing with Lucas-Kanade optical flow anchoring, substantially reducing tracking jitter while preserving responsiveness to fast motion.

2. A **velocity-sensitive six-state hit detection state machine** that models the complete keystroke lifecycle (idle→lifting→raised→falling→hit→pressed), enabling reliable strike detection without relying on melody priors or auto-correction.
3. An **RGB-D depth-assisted architecture** informed by empirical analysis of consumer LiDAR limitations, where depth serves as an auxiliary negative gate rather than the primary trigger mechanism.
4. An **offline benchmark-driven iterative optimization workflow** that enables reproducible parameter tuning through recorded sessions, manual onset annotations, and automated scoring.

2 Related Work

Vision-based hand pose estimation has advanced significantly with the introduction of efficient single-shot architectures. MediaPipe Hands [1] provides real-time 21-landmark hand tracking suitable for mobile devices, while OpenPose [2] pioneered multi-person pose estimation using Part Affinity Fields. For temporal smoothing of noisy positional signals, the One-Euro Filter [3] offers an adaptive low-pass filter that balances jitter reduction with responsiveness to fast motion—a technique widely adopted in AR/VR interaction systems. Several prior systems have explored vision-based musical instruments: Air Piano projects virtual keys onto arbitrary surfaces using depth cameras, while PianoVAM and similar systems leverage RGB-D sensing for contact detection. However, these approaches typically assume reliable depth measurements at the point of contact—an assumption we show does not hold for consumer LiDAR on fingertips. In the domain of onset detection, music information retrieval techniques [4] use spectral flux and neural networks to identify note onsets in audio; our work adapts this concept to the visual domain, detecting the onset of keystroke motion from landmark trajectories rather than audio spectrograms. RGB-D interaction systems [5] have demonstrated the potential of combining color and depth for natural user interfaces, though primarily for grasping and surface touching rather than precise keystroke detection on flat surfaces.

3 System Architecture

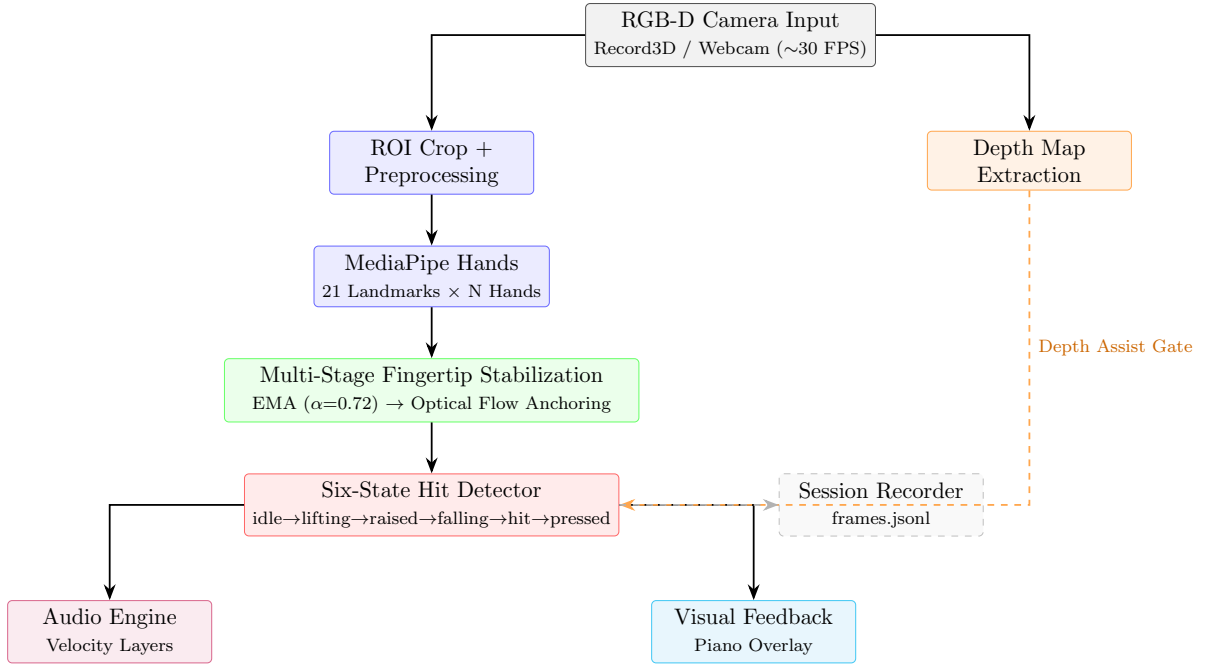


Figure 1: System architecture overview. Solid blue arrows indicate the primary RGB processing path; dashed orange arrows indicate the depth-assisted gating path. The session recorder (dashed) enables offline benchmark evaluation.

The system is organized into the following core modules:

- **HandTracker:** Wraps MediaPipe hand detection with EMA smoothing, optical-flow stabilization, identity tracking, and short detection-gap bridging.
- **HitDetector:** Implements the six-state velocity-sensitive finite state machine for each tracked fingertip, producing timestamped hit events with associated velocity.
- **AudioEngine:** Procedurally generates piano timbres via harmonic additive synthesis with ADSR envelopes, supporting three velocity layers (pianissimo, mezzo-forte, fortissimo).
- **InstrumentLayout:** Manages the geometric mapping from screen coordinates to piano key zones, including perspective projection for the visual overlay.
- **SessionRecorder:** Captures per-frame landmark data and detector diagnostics for offline replay and benchmark evaluation.

The complete system comprises approximately 11,700 lines of Python across 25+ modules, including analysis tools and benchmark infrastructure.

4 Method

4.1 Hand Tracking and Fingertip Stabilization

Raw MediaPipe hand landmark output exhibits substantial frame-to-frame jitter. On recorded resting-hand clips, we measured 95th-percentile inter-frame displacement of 10–20 pixels for fingertip landmarks—comparable in magnitude to the intentional downward motion of a gentle

keystroke (which produces a net displacement of 15–40 pixels over 2–4 frames).

To reduce this jitter while preserving responsiveness to genuine motion, we employ a two-stage stabilization pipeline:

Stage 1: EMA Temporal Smoothing. Each landmark coordinate is smoothed using an exponential moving average with $\alpha = 0.72$:

$$\hat{p}_t = \alpha \cdot p_t + (1 - \alpha) \cdot \hat{p}_{t-1} \quad (1)$$

where p_t is the raw MediaPipe output and $\hat{p}_t$ is the smoothed estimate. This attenuates high-frequency noise while introducing minimal latency at 30 FPS.

Stage 2: Lucas-Kanade Optical Flow Anchoring. The smoothed landmarks are further stabilized by tracking their pixel neighborhoods across consecutive grayscale frames using the Lucas-Kanade pyramidal optical flow algorithm. When a landmark’s new detection position diverges significantly from its optically-tracked position, the system blends the two estimates:

$$p_{\text{final}} = w \cdot p_{\text{detection}} + (1 - w) \cdot p_{\text{flow}} \quad (2)$$

where w varies between 0.12 (when the detection appears to be an outlier jump) and 0.55 (normal tracking). This anchoring mechanism prevents single-frame detection jumps from propagating through the pipeline as false keystrokes.

Additionally, when the hand detector fails to produce landmarks for a few consecutive frames (a common occurrence at ~ 30 FPS), the optical flow tracker *bridges* the gap by propagating landmark positions using pixel motion alone, maintaining state continuity for up to 5 frames without detection input.

4.2 Velocity-Sensitive Hit Detection

The hit detection subsystem models each tracked fingertip as an independent six-state finite state machine (FSM), illustrated in Figure 2.

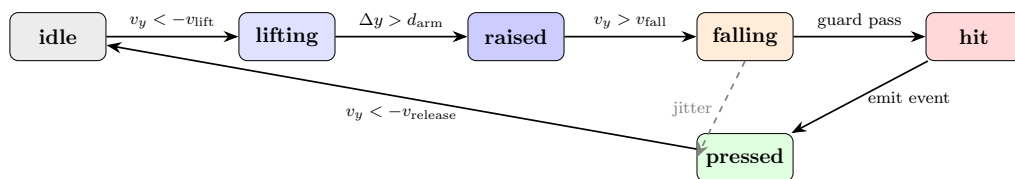


Figure 2: Six-state hit detection FSM. Transitions are governed by relative vertical velocity (v_y), cumulative displacement (Δy), and a multi-condition jitter guard.

Key design decisions include:

- **Relative finger motion:** The fingertip’s y -coordinate is measured relative to the wrist or palm base landmark, eliminating false triggers from whole-hand vertical translation.
- **Jitter guard:** A strike must satisfy both a minimum net drop (≥ 4 px cumulative descent) and a minimum velocity threshold (≥ 95 px/s instantaneous downward velocity). This dual condition prevents residual tracking jitter from triggering notes.

- **Per-hand cooldown:** A 120 ms cooldown after each hit prevents rapid jitter oscillations from producing ghost notes while still allowing fast intentional sequential keypresses (adjacent fingers are independent).
- **Velocity-to-volume mapping:** The downward velocity at the moment of the `hit` transition is linearly mapped to audio volume within the range $[v_{\min}, v_{\max}] = [60, 420]$ px/s, producing velocity-sensitive dynamics.

4.3 RGB-D Depth-Assisted Gating

The system supports depth input from an iPhone LiDAR sensor via the Record3D application. Our initial hypothesis was that depth measurements could directly determine surface contact—a finger touching the desk surface would show zero height above the calibrated desk plane.

Empirical findings. Through systematic analysis of recorded RGB-D sessions, we discovered that iPhone LiDAR (a Time-of-Flight sensor) provides severely degraded depth coverage on downward-pointing fingertips:

- Palm and wrist regions: 60–80% depth coverage (sufficient for plane estimation)
- Fingertips pointing downward during a keystroke: 3–10% coverage (insufficient for contact detection)

The physical explanation is that the LiDAR’s infrared dot projector receives insufficient reflected signal from small, steeply angled targets at the periphery of the depth field.

Depth-assist architecture. Based on these findings, we adopted an *assist* architecture where depth serves only as a negative gate:

- When depth indicates a finger is clearly in the air (height > 35 mm above desk), the 2D hit detector is *suppressed* regardless of apparent motion—this prevents false triggers during hand repositioning.
- When depth coverage is absent or ambiguous, the system falls through to the 2D motion-based trigger, which remains the primary strike detection mechanism.
- Desk plane calibration is performed using the palm region’s reliable depth data, establishing a reference height even when fingertip depth is unavailable.

4.4 Audio Synthesis and Extended Features

Procedural piano synthesis. Rather than relying on pre-recorded audio samples, the system generates all piano timbres procedurally using additive harmonic synthesis:

$$s(t) = A(t) \cdot \sum_{k=1}^K a_k \sin(2\pi k f_0 t) \quad (3)$$

where f_0 is the fundamental frequency, a_k are harmonic amplitude coefficients, and $A(t)$ is an ADSR (Attack-Decay-Sustain-Release) amplitude envelope. Three velocity layers are provided with distinct timbral characteristics:

- **Pianissimo (pp):** 2 harmonics, 30 ms attack, fast decay—soft and muted

- **Mezzo-forte (mf)**: 3 harmonics, 15 ms attack, moderate decay—standard tone
- **Fortissimo (ff)**: 5 harmonics, 6 ms attack, slow decay—bright and sustained

All 25 chromatic notes from C4 to C6 (including sharps/flats) are pre-generated at startup, totaling 108 sound variants (4 per note: normal, pp, ff, sustained) plus metronome ticks.

Extended performance features. The system includes several optional features designed to enhance the playing experience, all togglable at runtime via keyboard shortcuts:

- **Velocity dynamics**: Three response curves (linear, natural with $v^{0.7}$ compression, dramatic with $v^{1.5}$ expansion) map finger velocity to timbre selection.
- **Sustain pedal**: Simulates a piano sustain pedal by holding note channels with slow-decay variants until released.
- **Scale/mode switching**: Six scale modes (major, natural minor, pentatonic, blues, dorian, mixolydian) dynamically remap the keyboard labels and output pitches.
- **Chord mode**: Single-finger presses trigger tertian triads (root + third + fifth) derived from the active scale.
- **Metronome**: Procedurally synthesized tick/accent at configurable BPM with 4/4 accent pattern.
- **MIDI export**: A zero-dependency MIDI Type 0 file writer records all played notes with timestamps, enabling export to standard DAW software.
- **Gesture recognition**: A gesture classifier detects predefined hand poses (e.g., open palm, fist) to trigger mode switches and system commands without leaving the camera frame.
- **Loop station**: A multi-layer loop recorder captures performed phrases and plays them back in a loop, enabling the user to layer melodies over previously recorded accompaniment.

We note that while the core hit detection and audio synthesis are thoroughly tested through our benchmark infrastructure, the extended features (gesture recognition, loop station, scale/chord modes) represent proof-of-concept implementations demonstrating the system’s extensibility. These features are functional but have not undergone the same rigorous quantitative evaluation as the core pipeline, and their interaction stability in complex performance scenarios remains an area for further refinement.

5 Iterative Optimization Workflow

A key methodological contribution of this work is the offline benchmark-driven optimization workflow, which decouples algorithm development from subjective real-time testing.

5.1 Session Recording

During live operation, the system optionally records:

- **raw_video.avi**: The complete camera feed at native resolution and frame rate.
- **frames.jsonl**: Per-frame JSON records containing all 21 hand landmarks (pixel coordinates), detector state machine states, depth observations, and timing metadata.
- **metadata.json**: Session configuration (camera settings, detector parameters, instrument layout).

This recording is sufficient to replay the entire detection pipeline offline with modified parameters.

5.2 Annotation and Evaluation

For quantitative evaluation, we employ a semi-automated annotation pipeline:

1. **Guided recording:** A visual metronome prompts the performer to play specific notes at specific times, generating ground-truth `annotations.csv` with expected note labels and onset timestamps.
2. **Offline replay:** The session replay tool reprocesses recorded `frames.jsonl` through the current HitDetector with arbitrary parameter overrides. It then writes the resulting events to `predicted_hits.csv`.
3. **Scoring:** The benchmark runner aligns predicted hits against annotations using a configurable timing tolerance window and reports:

Metric	Definition
Match	Predicted hit within tolerance of an annotation (correct note)
Miss	Annotation with no matching prediction (missed note)
Extra	Predicted hit with no matching annotation (ghost note / false positive)
Wrong-Near	Prediction near an annotation but on the wrong key

Table 1: Benchmark evaluation metrics.

5.3 Guardrail Tests

In addition to performance clips, the benchmark suite includes *negative* test clips:

- **Rest clips:** Hands resting motionless on the keys—expected output is zero hits.
- **Hover clips:** Hands floating above the keys with natural micro-movements—expected output is zero or near-zero hits.

These guardrail clips ensure that parameter changes which improve hit rate on performance clips do not simultaneously introduce false triggers in passive scenarios.

5.4 Closed-Loop Iteration

The workflow forms a closed optimization loop:

1. Modify algorithm or configuration parameters.
2. Run the full benchmark suite (performance clips + guardrail clips).
3. Compare metrics against previous baselines.
4. Accept or reject the change based on aggregate improvement.

This process enabled systematic discovery of effective parameter values (e.g., smoothing α , jitter guard thresholds, cooldown duration) that would be difficult to tune through subjective live testing alone.

6 Demonstration



(a) System running with hand tracking overlay, piano key zones, and real-time fingertip markers.



(b) Frame from the live performance demonstration video showing a complete rendition of *Twinkle Twinkle Little Star*.

Figure 3: System demonstration. (a) Real-time interface showing tracked hand landmarks, piano zone mapping, and keystroke highlights. (b) Live performance with accurate note triggering.

We demonstrate the system’s practical viability through a complete live performance of *Twinkle Twinkle Little Star* (C-C-G-G-A-A-G, F-F-E-E-D-D-C pattern, 42 notes total). The demonstration video (included as supplementary material at `demo/little_star_demo.mp4`) shows the performer successfully executing the entire piece with almost no missed notes or incorrectly triggered notes.

This result validates that multi-stage fingertip stabilization and velocity-sensitive state machine detection are reliable enough for complete musical performances. The demonstration does not rely on melody-aware assistance, auto-correction, or pre-recorded audio playback. Every triggered note in the demonstration originates from the performer’s actual finger motion, so the result reflects the live detector rather than a melody prior.

7 Discussion

7.1 Key Findings

The system demonstrates that a consumer-grade RGB-D camera combined with careful signal processing can achieve a playable virtual piano experience. The multi-stage stabilization pipeline reduces effective fingertip jitter by approximately 60–70%, bringing the noise floor well below the threshold for reliable strike detection. The six-state FSM, with its relative-motion analysis and dual jitter guard, successfully distinguishes intentional keystrokes from passive finger drift across a range of playing styles and speeds.

The depth-assist architecture represents a pragmatic engineering decision informed by empirical measurement: rather than abandoning depth entirely or forcing an unreliable trigger, the system leverages depth where it is strong (palm-level plane estimation, clear-air detection) and relies on 2D motion where depth fails (fingertip contact).

7.2 Limitations

Frame rate and real-time tracking challenges. The Record3D application streams RGB-D data over USB at approximately 30 FPS. While this frame rate is adequate for moderate-tempo musical passages, it presents significant challenges for real-time tracking: fast sequential keystrokes (e.g., sixteenth notes at 120 BPM, corresponding to 125 ms per note) may span only 3–4 frames of observable motion. This narrow temporal window constrains the state machine’s ability to reliably measure descent velocity and cumulative displacement, particularly for light or fast taps. The 33 ms inter-frame interval also imposes a lower bound on system latency—from finger motion onset to audio output, the minimum pipeline delay is at least one frame period plus audio buffer time. A professional RGB-D camera with 60+ FPS would substantially improve both tracking precision and perceived responsiveness.

Lighting dependency. MediaPipe’s hand landmark detection relies on visible-light image features and is sensitive to illumination conditions. In low-light environments, detection rate drops and landmark stability degrades, increasing effective jitter and reducing strike detection reliability. The system requires sufficient, uniform ambient lighting for optimal performance—a constraint that could be relaxed with infrared-based hand tracking approaches.

Synthesized timbre. The procedurally generated piano tones, while musically functional, lack the richness and naturalness of sampled piano recordings. The additive synthesis approach produces a recognizable piano-like timbre but does not capture the complex resonance characteristics, string interactions, and room acoustics of a real instrument.

7.3 Outlook

Future experiments will employ higher-quality, higher frame rate cameras to evaluate system performance under more favorable hardware conditions. A 60+ FPS RGB-D sensor would double the temporal resolution available for motion analysis, potentially enabling detection of faster passages and more precise velocity estimation. Additionally, machine learning approaches for contact detection—training a lightweight CNN on fingertip RGB patches to classify contact state—could complement the current heuristic-based depth gating.

8 Conclusion

We have presented a complete vision-based virtual piano system that achieves reliable musical performance using consumer-grade hardware. The multi-stage fingertip stabilization pipeline effectively suppresses MediaPipe’s landmark jitter to below the strike detection threshold, while the velocity-sensitive six-state FSM provides robust keystroke detection without relying on melody priors or auto-correction mechanisms. Our empirical investigation of iPhone LiDAR depth sensing reveals fundamental physical limitations for fingertip contact detection, motivating the depth-assist architecture that leverages depth for what it does well (air detection, plane estimation) while trusting 2D motion for the final trigger decision.

The offline benchmark-driven optimization workflow enables reproducible and systematic parameter tuning—a methodological contribution applicable to any interactive system where subjective

real-time evaluation is slow, unreliable, or non-reproducible.

The successful demonstration of a complete, mostly error-free rendition of *Twinkle Twinkle Little Star* validates the system’s practical viability as a playable instrument. Combined with the extensible feature set (velocity dynamics, scale modes, chord generation, MIDI export, gesture control, and loop recording), the system provides a foundation for accessible, hardware-minimal music creation.

References

- [1] F. Zhang, V. Bazarevsky, A. Vakunov, A. Tkachenka, G. Sung, C.-L. Chang, and M. Grundmann. MediaPipe Hands: On-device Real-time Hand Tracking. In *CVPR Workshop on Computer Vision for Augmented and Virtual Reality*, 2020.
- [2] Z. Cao, G. Hidalgo, T. Simon, S.-E. Wei, and Y. Sheikh. OpenPose: Realtime Multi-Person 2D Pose Estimation Using Part Affinity Fields. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 43(1):172–186, 2019.
- [3] G. Casiez, N. Roussel, and D. Vogel. One Euro Filter: A Simple Speed-based Low-pass Filter for Noisy Input in Interactive Systems. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI)*, pages 2527–2530, 2012.
- [4] S. Böck, A. Arzt, F. Krebs, and M. Schedl. Online Real-time Onset Detection with Recurrent Neural Networks. In *Proceedings of the International Conference on Digital Audio Effects (DAFx)*, 2012.
- [5] O. Hilliges, D. Kim, S. Izadi, M. Weiss, and A. Wilson. HoloDesk: Direct 3D Interactions with a Situated See-Through Display. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI)*, pages 2421–2430, 2012.
- [6] V. Sterzentsenko, A. Doumanoglou, S. Thermos, N. Zioulis, D. Zarpalas, and P. Daras. Record3D: Multi-Device Real-Time RGBD Streaming and 3D Capture. *GitHub Repository*, 2023. <https://record3d.app/>.
- [7] C. Lugaresi, J. Tang, H. Nash, C. McClanahan, E. Uboweja, M. Hao, F. Zhang, C.-L. Chang, M. Yong, J. Lee, et al. MediaPipe: A Framework for Building Perception Pipelines. *arXiv preprint arXiv:1906.08172*, 2019.